

『生无所息，斗无所止』



HUST Artificial Intelligence Robot laboratory

哈尔滨理工大学AIR人工智能机器人创新实验室

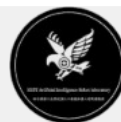
电控组第二讲

培训时间：第 7 周 10.17 主讲人：_____

关于时间安排：



周次与日期	培训内容	本节重点	课后成果
军训培训 9月25日	C语言①：环境配置与基础语法	编译运行过程、变量、数据类型、常量、运算符、输入输出、类型转换	能独立创建工程并完成简单计算程序
第7周 10月17日	C语言②：程序流程与函数，数组、字符串、结构体与指针	if、switch、for、while、函数、参数、返回值、作用域、数组下标、字符串结尾、结构体组织数据、地址与指针、指针传参	掌握分支语句、结构体、指针等高阶语句，初步掌握C语言
第8周 10月24日	STM32①：GPIO、按键与外部中断	工程建立、GPIO输入输出、LED、上下拉、EXTI、按键消抖	完成按键控制LED，并能处理按键抖动
第9周 10月31日	STM32②：定时器、PWM	定时器周期计算、非阻塞闪灯、PWM频率与占空比、舵机脉宽控制	完成PWM控制LED呼吸灯与舵机角度
第10周 11月7日	STM32③：UART与I ² C OLED	串口收发、字符串转数字、命令解析；OLED初始化和数据显示	串口控制频率/占空比/舵机角度，OLED显示系统状态
第11周 11月14日	正式考核	独立完成、录制视频演示、代码内容讲解	在安排的规定考核时段内完成，周日前提交



目录

01

程序流程与分支

02

循环语句

03

函数

04

数组、字符串、结构体与指针

『生无所息，斗无所止』



HUST Artificial Intelligence Robot laboratory

哈尔滨理工大学AIR人工智能机器人创新实验室

01 程序流程与分支



1. 问题：重复的代码只能靠手抄

要让一个 LED 闪十次，按顺序写就得抄十遍。

```
/* 顺序：一条直线 */  
a = 3; b = 4; c = a + b;
```

2. 分支：条件不同，走不同的路

引脚读到低电平，才去消抖并把状态翻过来。

```
/* 分支：满足条件才走 */  
if (a > b) c = a;
```

3. 循环：同一段代码重复执行

重复到条件不成立为止，闪灯、扫描、遍历都靠它。

```
/* 循环：重复到条件不成立 */  
for (i = 0; i < 3; i++) c = c + i;
```

本页判断：程序默认是一条直线；分支决定走哪条路，循环决定一段路走多少遍。



1. 条件真假：0 为假，其他都为真

C 判断的是「括号里表达式的值是不是 0」。

2. else if 梯：命中一个就停

从上往下依次判断，第一个成立的执行完就结束。

3. 一条语句也别省大括号

省掉以后补一行，逻辑就悄悄变了，统一写上。

```
/* 条件真假：0 为假，非 0 为真 */  
if (flag) { ... } /* flag != 0 */  
if (!flag) { ... } /* flag == 0 */  
/* else if 梯：命中一个就跳出 */  
if (score >= 90) grade = 'A';  
else if (score >= 60) grade = 'P';  
else grade = 'F';
```

本页判断：if 判断的是括号里的值是不是 0；else if 从上往下命中即停；大括号一律写上。



1. switch 只认整数和字符

case 后面必须是常量，不能写区间、不能比字符串。

2. break 是每个分支的出口

忘了写就会穿透，接着执行下面那个 case。

3. default 兜住没命中的情况

命令解析里，它就是「不认识这条命令」那一路。

```
/* switch：按值选一条路走 */
switch (cmd) {
case 'A': LED_ON(); break;
case 'B': LED_OFF(); break;
default: break;
}
/* 忘了 break：会一路穿透 */
case 'A': LED_ON();
case 'B': LED_OFF(); /* 发 A 也关灯 */
```

本页判断：选项少用 if；选项多、且都是整数或字符，用 switch，每个 case 记着写 break。

『生无所息，斗无所止』



HUST Artificial Intelligence Robot laboratory

哈尔滨理工大学AIR人工智能机器人创新实验室

02 循环语句



1. 三段式的分工

初始化只做一次；判断不成立就退出；更新在每轮之后。

2. 最适合「次数已知」

闪十次、取数组前 n 个，写成 for 一眼看出次数。

3. 循环变量只在循环里活

在 for 里定义 i，出了循环就不能再引用它。

```
/* 三段式：初始化 ; 判断 ; 更新 */  
for (int i = 0; i < 5; i++) {  
    LED_Toggle();  
    Delay(200);  
}  
/* 实际执行顺序 */  
/* 初始化 → 判断 → 循环体 → 更新 → 回判断 */  
/* i 取 0,1,2,3,4 时成立，循环体共 5 次 */
```

本页判断：次数已知的重复用 for；初始化只做一次，判断、循环体、更新一轮一轮转。



循环	判断时机	适合的场景	至少跑一次
for	每轮开始之前	次数已知：闪十次、取数组前 n 个	否
while	每轮开始之前	次数未知、靠条件停：等待按键按下	否
do-while	每轮结束之后	必须先做一次再判断：菜单至少显示一次	是

语法 for (初始化 ; 判断 ; 更新) | while (判断) | do { 循环体 } while (判断)

本页判断：次数知道就用 for ，条件停就用 while ，必须先做一次就用 do-while 。



写法	做了什么	用在哪	别搞混
break	立刻跳出整个循环	找到目标值就提前收工，不再往下找	双层循环只跳一层
continue	跳过本轮剩下的语句，进入下一轮	过滤掉不满足条件的项，继续处理后面的	它不结束循环

死循环三因

循环变量忘了更新 • 判断条件永远为真 • 循环体里改掉了判断用的变量

本页判断：break 是收工，continue 是换下一个；循环停不下来，先看变量有没有更新。

『生无所息，斗无所止』



HUST Artificial Intelligence Robot laboratory

哈尔滨理工大学AIR人工智能机器人创新实验室

03 函数



1. 函数 = 输入 + 处理 + 返回值

名字要说明它干什么，Add 比 f1 有用得多。

2. 定义和声明是两件事

声明先告诉编译器「有这个函数」，定义才写实现。

3. 返回值只能带回一个

要一次带回多个结果，得靠指针，下一页讲。

```
/* 定义：返回类型 函数名 ( 参数表 ) */  
int Add(int a, int b) {  
    return a + b; /* 交给调用者 */  
}  
/* 调用：传实参，接返回值 */  
int s = Add(3, 5); /* s = 8 */  
int t = Add(s, 10); /* 结果还能再用 */
```

本页判断：同一段代码出现第二次就切函数；名字说明作用，参数给全，返回值接住。



1. 值传递只改副本

函数里怎么改形参，外面的实参都不会动。

2. 指针传递传的是地址

想改外面，就传地址，函数里用 * 解引用改它。

3. 数组是那个例外

数组名传参就退化成指针，函数里改外面会跟着变。

```
/* 值传递：拿到的是副本 */
void badSwap(int x, int y) {
    int t = x; x = y; y = t; /* 白换 */
}
/* 指针传递：传地址，改原变量 */
void swap(int *x, int *y) {
    int t = *x; *x = *y; *y = t;
}
badSwap(a, b); /* a=1 b=2 没换成功 */
swap(&a, &b); /* a=2 b=1 换成了 */
```

本页判断：变量传参给的是复印件；想让函数改到外面的变量，必须把地址传进去。



1. 局部变量出了大括号就没了

在哪个大括号里定义，就只能在那里用。

2. 全局变量哪里都能碰

传状态很方便，但也哪里都能被改错，慎用。

3. static 让它活得久一点

函数里的 static 只初始化一次，退出函数值还在。

```
int g_cnt; /* 全局：都能用 */
void Count(void) {
    static int n = 0; /* 只初始化一次 */
    n++; /* 退出也不销毁 */
    int tmp = n; /* 局部：出函数就没了 */
    printf("n = %d\n", n);
}
/* tmp 出了函数就不能再用 */
```

本页判断：变量先看作用域再看生命周期；出大括号就没了，static 会一直活着。

『生无所息，斗无所止』



HUST Artificial Intelligence Robot laboratory

哈尔滨理工大学AIR人工智能机器人创新实验室

04 数组、字符串、结构体 与指针



1. 数组 = 一排同类型的格子

一次能装一组数，比如连续采 8 个电压值，就开 8 个格子。

2. 下标从 0 开始，到 n-1 结束

第 1 个元素是 `adc[0]`，第 8 个是 `adc[7]`，没有 `adc[8]`。

3. 越界不报错，但会改坏内存

这是最难查的一类 bug；元素个数用 `sizeof` 算，别用手数。

```
int adc[8]; /* 8 个 int，连续排在内存里 */
adc[0] = 100; /* 第 1 个：下标从 0 起 */
adc[7] = 200; /* 第 8 个：最后一个 */
adc[8] = 300; /* 越界！不报错，但改坏了别人的内存 */
```

本页判断：数组是一排同类型的格子；下标从 0 起，越界不报错但会出事。



1. 字符串 = char 数组 + 结尾的 '\0'

"AIR" 一共占 4 个字节：'A'、'I'、'R' 和收尾的 '\0'。

2. strlen 一路数到 '\0' 为止

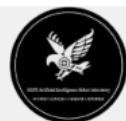
返回的是可见字符个数，不算结尾那个 '\0'。

3. sizeof 量的是整个数组

char buf[16] 的 sizeof 永远是 16，哪怕只装了 3 个字。

```
char name[16] = "AIR"; /* 3 个字符 + 结尾 '\0' */
strlen(name) -> 3; /* 数到 '\0' 为止 */
sizeof(name) -> 16; /* 数组多大就多大 */
char *p = "AIR"; /* 指针指向常量区，不能改 p[0] */
```

本页判断：字符串结尾必须留 '\0'；strlen 数字符个数，sizeof 量数组大小。



1. 结构体 = 自己定义的数据类型

把 year、month、day 打包成 Date，之后就能像 int 一样用它。

2. 变量用点，指针用箭头

d.month 是变量访问；p->month 等价于 (*p).month。

3. 它经常和数组一起出现

Battery pack[4] 就是 4 组电池的数据，一行装一类信息。

```
typedef struct { int y; int m; int d; } Date; /* 定义新类型 */  
Date d = { 2026, 10, 17 }; /* 定义变量并初始化 */  
d.m = 12; /* 用点取成员 */  
Date *p = &d; p->y = 2027; /* 指针用箭头取成员 */
```

本页判断：结构体把相关的数据捆成一个整体；变量用点，指针用箭头。



1. 指针里装的是地址

`int *p = &a;` 里，`p` 装的是 `a` 的内存位置，不是 10。

2. & 取地址，* 去取东西

`p` 的意思是「到 `p` 指的那个地址去」，可以读，也可以写。

3. 指针的类型决定走多大一步

`int *` 走一步 4 字节，`char *` 走一步 1 字节；数组名就是首元素地址。

```
int a = 10;
int *p = &a; /* p 里装的是 a 的地址 */
p = 20; /* 解引用：改的就是 a 本身 */
int *q = arr; /* 数组名 arr 就是首元素地址 */
```

本页判断：指针是装地址的变量；& 取地址，* 到那个地址取东西。



1. 需求：一次换掉两个变量

return 只能带回一个值，两个都换就得靠指针写回去。

2. 值传递版本为什么白换

形参 x、y 是副本，函数里换的是副本，外面 a、b 没动。

3. 指针版本就三步

存一份副本、写过去、写回来，交换就完成了。

```
/* 指针版 swap：真的能换 */  
void swap(int *x, int *y) {  
    int t = *x; /* 存一份 */  
    x = *y; /* 写过去 */  
    y = t; /* 写回来 */  
}  
int a = 1, b = 2;  
swap(&a, &b); /* a=2 b=1 */
```

本页判断：函数要把结果写到外面，参数就得是指针，调用时传地址。



1. 计算日期是当年的第几天

定义一个结构体 `Date`，包含年、月、日三个成员。输入一个日期，编程计算该日期是该年中的第几天。提示：考虑闰年情况。

2. 交换两个变量的值（使用指针）

编写一个函数 `swap(int *x, int *y)`，使用指针交换两个整数的值，并在主函数中测试。

3. 求数组元素平均值

先输入需要输入整数的个数 `n`，再输入 `n` 个整数（用数组存储），计算并输出这些数的平均值（保留两位小数）。

4. 函数指针调用演示

编写一个函数 `int add(int a, int b)` 返回两数之和，在主函数中使用函数指针调用该函数并输出结果。

本页判断：四道题覆盖结构体、指针、数组和函数指针，先在本地跑通再上传。



1. 注释统一用 UTF-8

在 Keil / VSCode 里确认编码设置，中文注释乱码会直接扣分。

2. 代码能编译、能跑出结果

提交前自己跑一遍，别把编译不过的代码交上来。

3. 一个压缩包装齐材料

演示视频、代码和其他材料放在同一个压缩包里。

4. 命名按「班级 + 姓名 + 第几次任务」

例如：电气 26-5 班 + 李华 + 第一次任务 .zip；以附件形式发送到指定邮箱，邮件主题同名。

5. 不强制每周交，但考核前必须交齐

转入梯队考核上传结束时间前完成全部上传，逾期不再补收。

本页判断：编码 UTF-8、代码能跑、材料齐、命名规范、期末交齐，五条都做到再上传。

『生无所息，斗无所止』



HUST Artificial Intelligence Robot laboratory

哈尔滨理工大学AIR人工智能机器人创新实验室

下课啦!